Google TabFM Implementation Handbook

1. Purpose

This handbook is the operational manual for the `google-tabFM-implementation` repository. It explains what the project does, how it is structured, how to run it safely, what outputs to expect, and how to debug common failures.

This document is written from repository evidence: - source code in `src/` and `scripts/`, - executed notebooks in `notebooks/` and `problems/`, - persisted artifacts in `problems/*/artifacts/`, - local verification commands executed in this environment.

---

2. Glossary

- **TabFM**: Google tabular foundation model APIs used through the `tabfm` package.
- **Checkpoint**: Serialized model weights (`pytorch_model.bin`) used by TabFM loaders.
- **Benchmark portfolio**: Multi-dataset benchmark defined in `src/tabfm_benchmark/datasets.py`.
- **E2E**: End-to-end notebook execution plus artifact validation.
- **Context cap**: Maximum rows used for TabFM fit via `TABFM_CONTEXT_MAX_ROWS`.
- **Eval cap**: Maximum evaluation rows for comparisons via `TABFM_EVAL_MAX_ROWS`.
- **Fast mode**: Lighter runtime mode controlled by `TABFM_FAST_MODE=1`.
- **Champion model**: Best model selected in each notebook and written into runtime metadata JSON.
- **Threshold curve**: Sweep of score cutoff vs policy/business outcomes.
- **Top-k campaign**: Policy targeting top proportion of high-risk users.

---

3. Project Scope and Objectives

3.1 What this repository contains

1. A reusable Python benchmark package for TabFM (`src/tabfm_benchmark`).
2. A strict notebook execution orchestrator with retry/backoff logic (`scripts/run_strict_e2e.py`).
3. Eight production-style problem notebooks under `problems/`.
4. Executed notebook outputs and persisted model/policy artifacts.

3.2 Engineering goals

- Reproducible tabular ML workflows with controlled splits and seeds.
- Honest comparison against strong tree baseline (XGBoost).
- Artifact-first workflow for traceability and downstream consumption.
- Runtime controls for constrained hardware.

---

4. Prerequisites

- Linux environment.
- Python 3.11+ (`pyproject.toml` specifies `>=3.11`).
- `uv` package manager.
- Optional GPU for faster runs (`TABFM_DEVICE=auto|cuda|cpu`).

4.1 Recommended runtime constraints (24 GB RAM discipline)

- Prefer `TABFM_FAST_MODE=1` in constrained environments.
- Use context and eval caps (`TABFM_CONTEXT_MAX_ROWS`, `TABFM_EVAL_MAX_ROWS`).
- Do not run all heavy notebooks concurrently.
- Keep `UV_CACHE_DIR=/tmp/uv-cache` when default cache path is read-only.

---

5. Environment Setup

```
uv venv --python 3.11
source .venv/bin/activate
uv sync --extra dev
```

5.1 Verified command behavior in this environment

The following command succeeded:

`UV_CACHE_DIR=/tmp/uv-cache uv run pytest`

Observed result: `- 7 passed, 1 warning in 2.30s`

Without `UV_CACHE_DIR`, `uv` failed due read-only cache location: `- Could not create temporary file ... /home/ahmad/.cache/uv/... Read-only file system`

---

6. Dependency Explanation

From `pyproject.toml` and notebook imports:

- `tabfm[pytorch]`: TabFM model APIs and PyTorch backend.
- `pandas`, `polars`: data processing and artifact serialization.
- `scikit-learn`: splits, preprocessing, metrics, some datasets (`fetch_openml`, built-in datasets).
- `xgboost`: baseline models.
- `jupyter`, `nbconvert`, `ipykernel`: notebook authoring and execution.
- `matplotlib`, `seaborn`: visualization in notebooks.

- `loguru`: structured notebook/script logging.
- `pytest`: unit/integration test suite for package behavior.

---

7. Folder Structure

```
.
|-- src/tabfm_benchmark/
|   |-- benchmark.py
|   |-- datasets.py
|   `-- types.py
|-- scripts/
|   |-- run_benchmark.py
|   `-- run_strict_e2e.py
|-- tests/
|   |-- test_schema.py
|   |-- test_metrics.py
|   |-- test_datasets.py
|   |-- test_integration_runner.py
|   `-- test_class_guard.py
|-- notebooks/
|   |-- tabfm_quickstart_benchmark.ipynb
|   `-- tabfm_telco_churn_production.ipynb
|-- problems/
|   |-- problem1_telecom_churn/
|   |-- problem2_saas_subscription_churn/
|   |-- problem3_credit_card_transaction_fraud/
|   |-- problem4_insurance_claim_fraud/
|   |-- problem5_house_price_prediction/
|   |-- problem6_ride_fare_estimation/
|   |-- problem7_loan_default_prediction/
|   `-- problem8_employee_attrition_prediction/
|-- data/
|   `-- raw/
|-- artifacts/
|-- README.md
|-- HANDBOOK.md
`-- pyproject.toml
```

---

8. Code Walkthrough

8.1 `src/tabfm_benchmark/types.py`

Defines `DatasetSpec` with: - `name`, `task_type`, `loader`, optional `row_cap`.

8.2 `src/tabfm_benchmark/datasets.py`

Contains default portfolio loaders: - classification: iris, wine, breast cancer, digits, covtype, - regression: diabetes, california housing.

8.3 `src/tabfm_benchmark/benchmark.py`

Core benchmark logic: - deterministic seed setup, - device validation, - stratified/standard splitting, - optional row capping, - TabFM estimator creation (`default` or `ensemble`), - metric calculation by task type, - stable result schema, - CLI interface via `main()`.

Also enforces class-cardinality guard (`MAX_SUPPORTED_CLASSES=10`).

8.4 `scripts/run_benchmark.py`

Thin entry script that delegates to `tabfm_benchmark.benchmark.main`.

8.5 `scripts/run_strict_e2e.py`

Notebook orchestrator: - builds per-problem config, - supports `--start-from` and `--only`, - retries on OOM signatures, - applies context backoff levels, - validates required artifacts, - writes reports: - `artifacts/strict_e2e_run_report.csv` - `artifacts/strict_e2e_run_report.md`

---

9. Training / Inference / Pipeline Flow

9.1 Benchmark package flow

1. Load default dataset specs.
2. Seed random generators.
3. Split train/test (stratified for classification).
4. Fit TabFM estimator.
5. Predict.
6. Compute metrics.
7. Persist full and summary outputs.

9.2 Problem notebook flow (common pattern)

1. Reproducible setup (seed, paths, env vars).
2. Data acquisition or cache loading.
3. Data cleaning + feature engineering.
4. Leakage-safe split strategy.
5. Train baseline XGBoost.
6. Train TabFM variants.
7. Compare model metrics (val/test).
8. Select champion model.

9. Optimize decision policy (threshold/top-k tiers).
10. Persist artifacts and runtime metadata.

9.3 Device/runtime behavior observed

- Several notebooks detect low GPU memory (<12 GiB) and fall back to CPU-safe training profiles.
- Context capping is actively used to keep fit stable in constrained runs.

---

10. Configuration and Environment Variables

Common variables found in notebooks and strict E2E runner:

10.1 Global TabFM controls

- `TABFM_DEVICE` (`auto|cpu|cuda`)
- `TABFM_CONTEXT_MAX_ROWS`
- `TABFM_EVAL_MAX_ROWS`
- `TABFM_FAST_MODE` (`0|1`)
- `TABFM_CHECKPOINT_PATH`

10.2 Problem-specific controls

- KKBox: `KKBOX_SAMPLE_TRAIN_ROWS`, `KKBOX_SAMPLE_EVAL_ROWS`, `KKBOX_MIN_SAMPLE_ROWS`
- Fraud: `FRAUD_DATA_URL`, `FRAUD_SAMPLE_TRAIN_ROWS`, `FRAUD_SAMPLE_EVAL_ROWS`, `FRAUD_MIN_SAMPLE_ROWS`
- Insurance: `INS_DATA_URL`, `INS_SAMPLE_TRAIN_ROWS`, `INS_SAMPLE_EVAL_ROWS`, `INS_MIN_SAMPLE_ROWS`
- House price: `HP_SAMPLE_TRAIN_ROWS`, `HP_SAMPLE_EVAL_ROWS`, `HP_MIN_SAMPLE_ROWS`, `OPENML_DATA_ID`
- Ride fare: `RIDE_DATA_URL`, `RIDE_SAMPLE_TRAIN_ROWS`, `RIDE_SAMPLE_EVAL_ROWS`, `RIDE_MIN_SAMPLE_ROWS`
- Loan: `LOAN_SAMPLE_TRAIN_ROWS`, `LOAN_SAMPLE_EVAL_ROWS`, `LOAN_MIN_SAMPLE_ROWS`
- Attrition: `ATTRITION_SAMPLE_TRAIN_ROWS`, `ATTRITION_SAMPLE_EVAL_ROWS`, `ATTRITION_MIN_SAMPLE_ROWS`
- E2E: `STRICT_E2E`

---

11. Commands Used (Verified)

11.1 Health checks run during this documentation pass

```
UV_CACHE_DIR=/tmp/uv-cache uv run pytest
UV_CACHE_DIR=/tmp/uv-cache uv run python scripts/run_benchmark.py --help
```

```
UV_CACHE_DIR=/tmp/uv-cache uv run python scripts/run_strict_e2e.py --help
free -h
ps -eo pid,ppid,pcpu,pmem,etime,cmd --sort=-pmem | head -n 15
gh auth status
```

11.2 Notable observed outputs

- Tests: `7 passed, 1 warning`.
- `run_benchmark.py --help`: confirms device/seed/estimator/output options.
- `run_strict_e2e.py --help`: confirms project selection options (`--start-from`, `--only`).
- `gh auth status`: active token is invalid, re-authentication required.

---

12. Validation / Evaluation / Metrics

12.1 Champion metrics from persisted artifacts

Problem	Champion	Test Metrics
problem1_telecom_churn	tabfm_ensemble_preset	ROC-AUC 0.8468, PR-AUC 0.6577, Accuracy 0.8020, F1 0.5854
problem2_saas_subscription	tabfm_default	ROC-AUC 0.9802, PR-AUC 0.8115, Accuracy 0.9545, F1 0.7191
problem3_credit_card_transactions	tabfm_default	ROC-AUC 0.7039, PR-AUC 0.4053, Accuracy 0.9970, F1 0.5714
problem4_insurance_claims	tabfm_ensemble_preset	ROC-AUC 0.7917, PR-AUC 0.5796, Accuracy 0.8000, F1 0.6341
problem5_house_price_prediction	tabfm_default	RMSE 24008.32, MAE 15397.57, R^2 0.8793, MAPE 9.52%
problem6_ride_fare_estimation	tabfm_ensemble_preset	RMSE 4.1677, MAE 2.2068, R^2 0.8343, MAPE 18.53%

Problem	Champion	Test Metrics
problem7_loan_default_prediction	tabpfn_tree_ensemble_preset	ROC-AUC 0.8318, PR-AUC 0.3887, Accuracy 0.9450, F1 0.3125
problem8_employee_attrition_prediction	tabpfn_preset	ROC-AUC 0.8449, PR-AUC 0.6399, Accuracy 0.8824, F1 0.5667

12.2 Policy output coverage

Policy summaries exist for problems 3-8, plus threshold summaries for 1,2,3,4,7,8 and top-k campaign files for 1,2.

---

13. Outputs and Interpretation

13.1 Key artifact types

- `*_model_metrics.csv`: model comparison tables for val/test.
- `*_predictions_test.parquet`: scored test predictions.
- `*_runtime_meta.json`: run configuration and champion metadata.
- `*_threshold_curve.csv`, `*_threshold_summary.csv`: policy threshold economics.
- `*_policy_summary.csv`, `*_topk_campaign.csv`, `*_actions.csv`: actionable decision summaries.

13.2 Example interpretation pattern

- If PR-AUC improves but policy net value degrades, threshold policy should be recalibrated.
- If runtime metadata shows aggressive context caps, metrics may reflect constrained-fit behavior rather than full-data potential.

---

14. Troubleshooting and Debugging

14.1 uv cache lock / read-only cache error

Symptom: `- Could not acquire lock ... /home/ahmad/.cache/uv/... Read-only file system`

Fix:

```
export UV_CACHE_DIR=/tmp/uv-cache
```

14.2 Notebook execution permission error in sandbox

Symptom from strict E2E logs: - `PermissionError: [Errno 1] Operation not permitted` during kernel socket startup.

Cause: - environment sandbox restrictions on networking/socket behavior for Jupyter kernel process.

Fix: - Run E2E in non-restricted local shell environment.

14.3 Missing TabFM checkpoint

Symptom from earlier strict E2E logs: - `FileNotFoundError: Weights not found at ... pytorch_model.bin`

Fix: - set `TABFM_CHECKPOINT_PATH` to a valid checkpoint, or allow notebook checkpoint download path to run.

14.4 GitHub publish auth failure

Symptom: - `gh auth status` reports invalid token.

Fix:

```
gh auth login -h github.com
```

---

15. End-to-End Execution Status

15.1 What is already available

- Executed notebooks exist for all 8 problems.
- Artifact folders per problem contain metrics, predictions, and runtime metadata.
- Strict E2E logs include both failure and successful runs for `problem1` across attempts.

15.2 What was verified in this pass

- Unit/integration tests and CLI health checks were run successfully.
- Existing artifacts were inspected and summarized.
- Full re-execution of all notebooks was intentionally not repeated to avoid unnecessary resource-heavy reruns while valid outputs already exist.

---

16. Deployment / Execution Notes

- This repo is best used as a **research/benchmark workflow** and artifact generator.

- For production deployment, package specific notebook logic into versioned pipeline modules and add CI checks for data contracts.
- Keep checkpoints and raw datasets in object storage or artifact registry; avoid storing large binary assets in Git.

---

17. Best Practices and Lessons Learned

1. Always keep a strong baseline (XGBoost) for grounded comparison.
2. Track policy outcomes, not only model metrics.
3. Use context/eval caps to control memory footprint under hardware constraints.
4. Persist runtime metadata to make results explainable and reproducible.
5. Separate raw data/model storage from source control for publish-ready repositories.
6. Include notebook automation (`run_strict_e2e.py`) with artifact validation for reliability.

---

18. Data and Storage Notes

Current local storage profile includes very large data/model assets (for example KKBox raw logs and TabFM checkpoint binaries). For public GitHub publishing, these paths are configured as local-only via `.gitignore`:

- `data/raw/`
- `problems/**/data/raw/`
- `problems/**/data/models/`

This keeps repository pushes reliable and avoids GitHub file-size violations.

---

19. References (Official / Source-of-Truth)

19.1 TabFM

- Google Research TabFM repository: https://github.com/google-research/tabfm
- TabFM model page (Hugging Face): https://huggingface.co/google/tabfm-1.0.0-pytorch
- Google Research announcement: https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/

19.2 Dataset endpoints used by notebook code

- Credit card dataset: https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv

- IBM attrition dataset: https://raw.githubusercontent.com/IBM/employee-attrition-aif360/master/data/emp_attrition.csv
- Insurance claims dataset: https://raw.githubusercontent.com/mwitiderrick/insurancedata/master/insura
- Taxis dataset: https://raw.githubusercontent.com/mwaskom/seaborn-data/master/taxis.csv
- Telco churn mirrors:
  - https://raw.githubusercontent.com/blastchar/telco-customer-churn/master/WA_Fn-UseC_-Telco-Customer-Churn.csv
  - https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/main/data/Telco-Customer-Churn.csv
  - https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv

19.3 Framework/API documentation

- PyTorch: https://pytorch.org/docs/stable/
- scikit-learn: https://scikit-learn.org/stable/
- XGBoost: https://xgboost.readthedocs.io/
- Polars: https://docs.pola.rs/
- pandas: https://pandas.pydata.org/docs/
- Jupyter: https://jupyter.org/documentation
- Kaggle API docs: https://www.kaggle.com/docs/api
- OpenML docs: https://docs.openml.org/

---

20. License Reminder

TabFM source code is Apache-2.0, but released TabFM weights used by notebook workflows are non-commercial licensed. Confirm downstream licensing before commercial deployment.